

Bioinformatics and Systems Biology Summer School 2026

<https://summeratmasaryk.cz/genes-in-motion-bioinformatics-and-systems-biology>

Genes in Motion

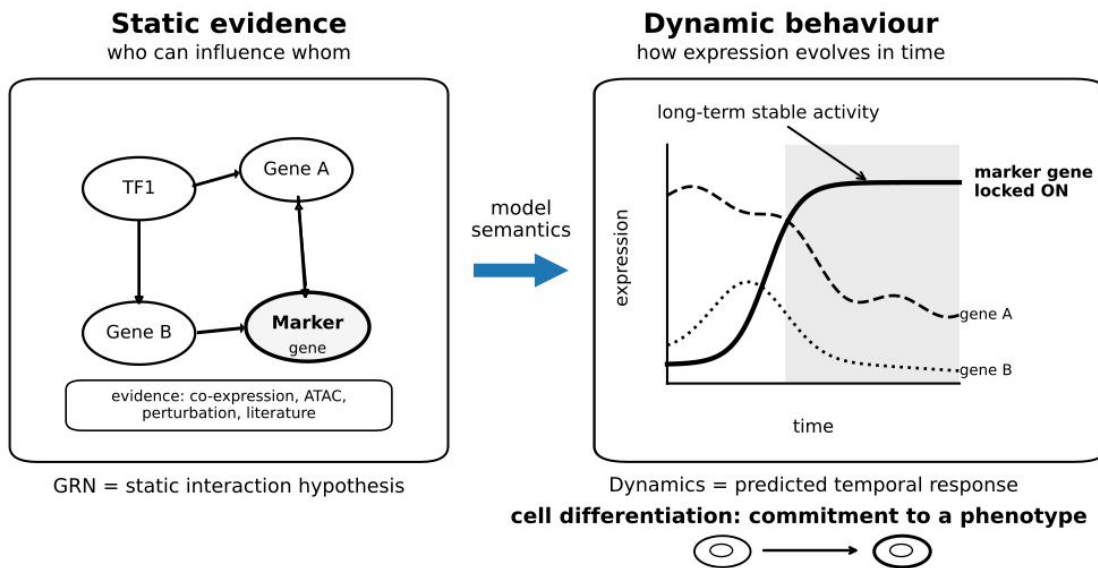
Modelling Dynamics of Gene Regulatory Networks with Boolean Networks

David Šafránek

Faculty of Informatics, Masaryk University, Brno

Motivation

- GRNs describe (statically) the evidence of mutual interactions among genes
- We need to predict how these interactions affect the gene expression in time
- Example from modelling cell differentiation:
 - important (marker) genes “lock” their activity for a significant amount of time (long term)

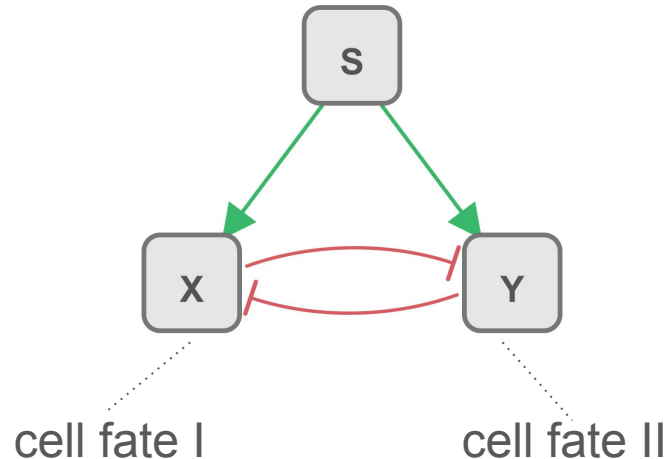


Example: Double Negative Feedback Loop

Gene X is a marker gene for establishing cell fate I

Gene Y is a marker gene for establishing cell fate II

Signal molecule S enables the cell fate decision



Experimental Observation

Assume all cells have no X and Y expressed

After some time the signal molecule S is added to the solution

Differentiation process is observed iff:

- Some cells start to express X

- Some cells start to express Y

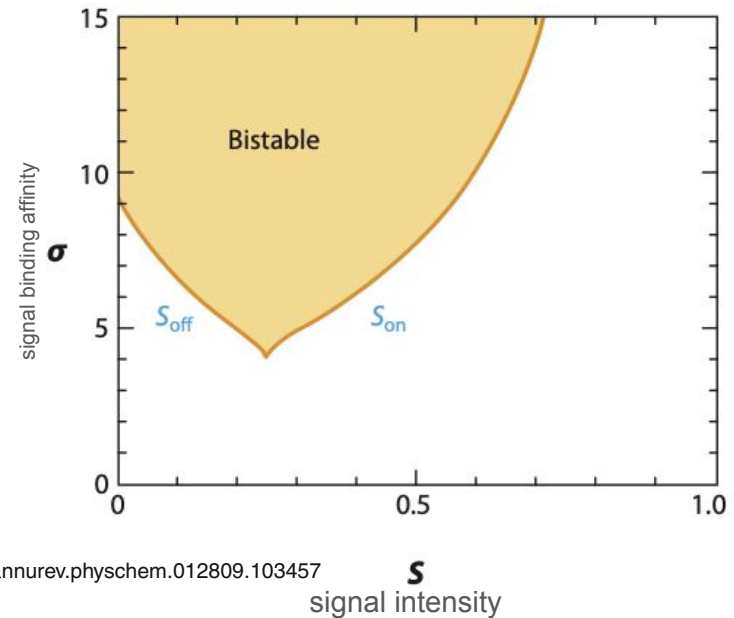
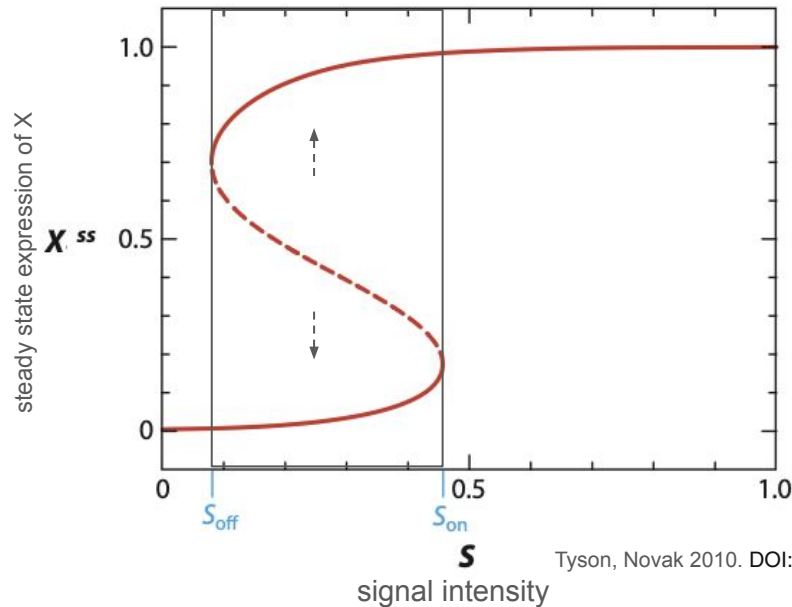
Detailed biochemical affinity of X and Y genes to the signal S decides which direction individual cells take

Final outcome of cell differentiation is observed in the long term

Notes from Dynamical Systems Theory

The possibility to choose if X goes to 0 or to 1 in long term depends on S

The “switch” curve is set by kinetic aspects of how S binds to promotor of gene X



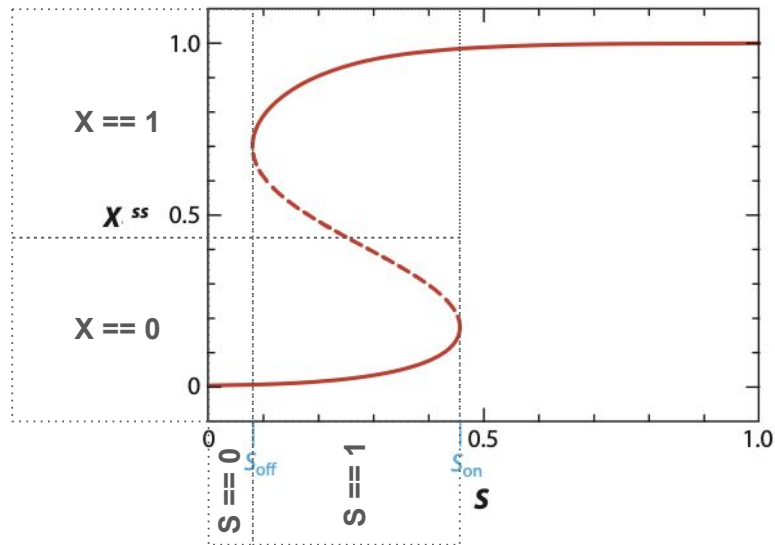
Boolean Networks

Unfortunately we do not know the kinetic details in most of the cases...

...so how to put GRNs into motion without that “biomechanistic” information?

The core idea of qualitative abstraction:

- Introduce qualitative (boolean) **states**



Boolean Networks

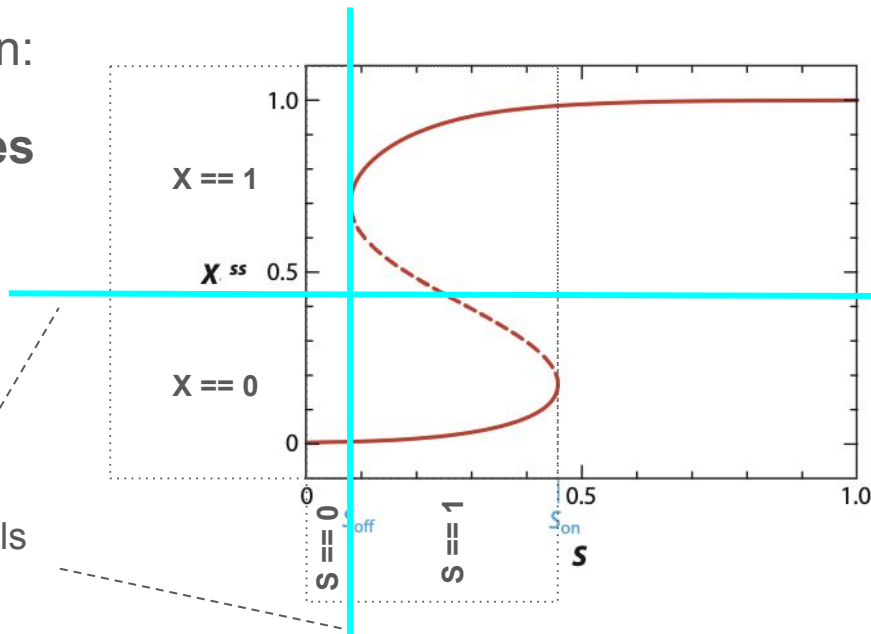
Unfortunately we do not know the kinetic details in most of the cases...

...so how to put GRNs into motion without that “biomechanistic” information?

The core idea of qualitative abstraction:

- Introduce qualitative (boolean) **states**

thresholds separating significant expression levels



Boolean Networks

Some states can be marked as **phenotypes**:

X expressed, Y not expressed $[X == 1, Y == 0]$ – *cell fate I*

X not expressed, Y expressed $[X == 0, Y == 1]$ – *cell fate II*

Mapping to experimental data – **discretisation**:

- assume some statistically significant threshold on read counts
 - change of the thresholds changes the understanding/interpretation of data
- interpretation of Boolean values is always based on several modelling assumptions

Boolean Networks

The concept of a **regulation** abstracts the *influence* of a gene on another gene

Some examples:

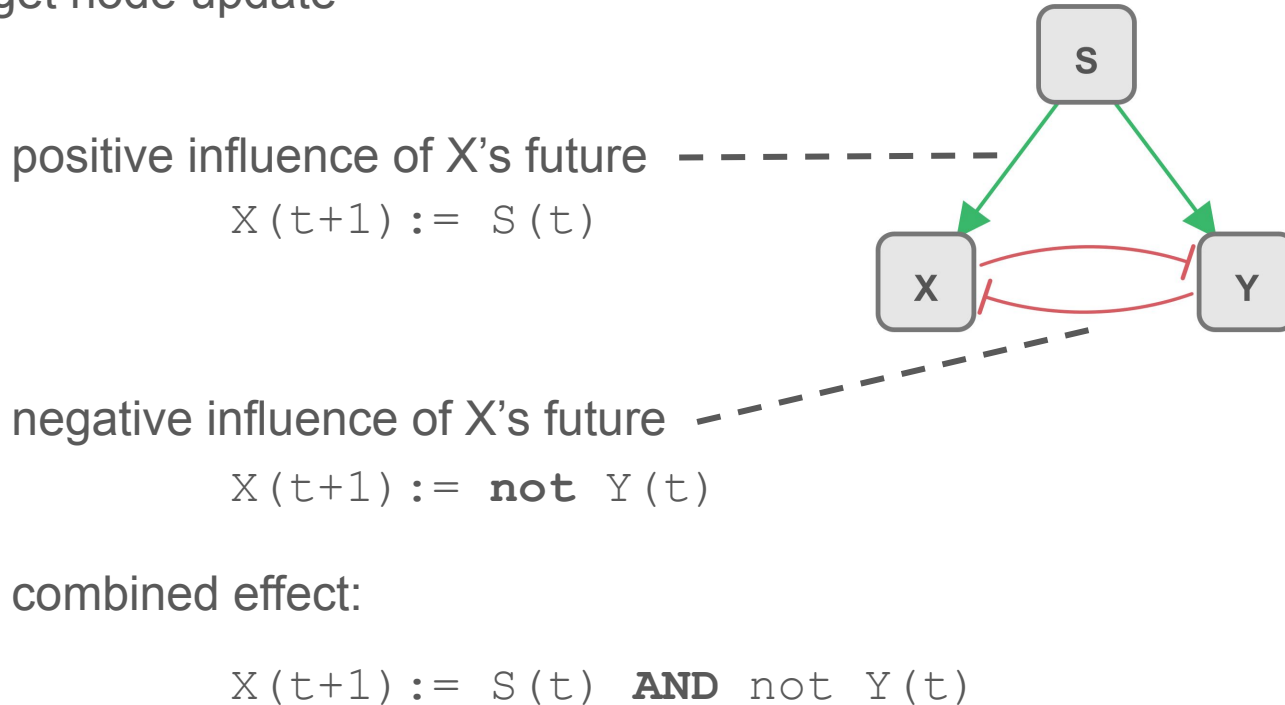
- transcription factor X binds to a promoter of gene Y leading to activation of X expression
- transcription factor X binds to a promoter of gene Y leading to deactivation of expression
- active signal molecule S binds to a transcription-binding complex of gene X causing activation of X expression

In any case regulations trigger *state changes*

- **transitions** between states

Meaning of Regulations

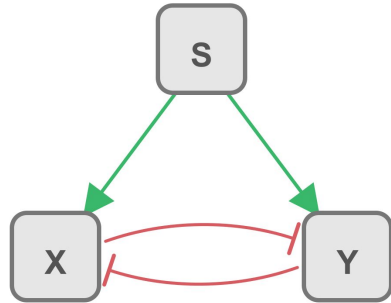
Target node update



Boolean Network Syntax

Boolean Network is a set of Boolean variables

Each variable is assigned a logic function describing it's update



$$X(t+1) := S(t) \textbf{ AND } \text{not } Y(t)$$

$$Y(t+1) := S(t) \textbf{ AND } \text{not } X(t)$$

$$S(t+1) := 1$$

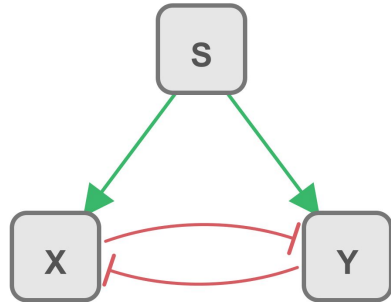
Input nodes (not regulated) are considered constant

(here we choose S to be constantly activated)

Boolean Network Syntax

Boolean Network is a set of Boolean variables

Each variable is assigned a logic function describing it's update



$$f_X(S, Y) := S \text{ **AND** not } Y$$

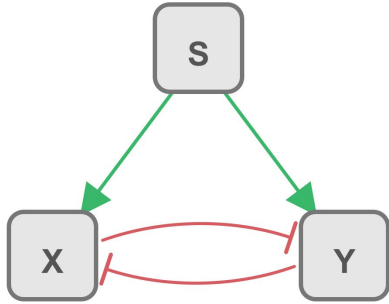
$$f_Y(S, X) := S \text{ **AND** not } X$$

$$f_S := 1$$

Input nodes (not regulated) are considered constant

(here we choose S to be constantly activated)

Syntax Matters!



`f_X(S, Y) := S AND not Y`

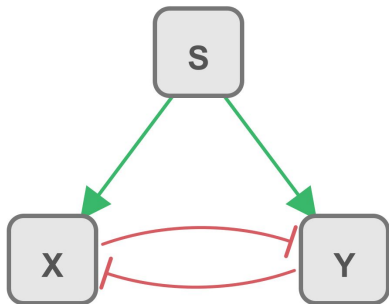
What happens if we write:

`f_X(S, Y) := S AND Y`

or

`f_X(S, Y) := S`

Syntax Matters!



$f_X(S, Y) := S \text{ **AND** not } Y$

What happens if we write:

$f_X(S, Y) := S \text{ **AND** } Y$

or

$f_X(S, Y) := S$

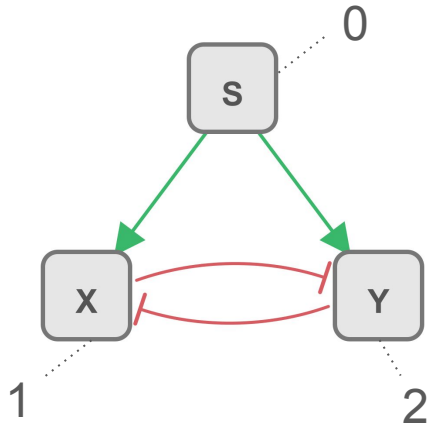
Logics is expected to be
CONSISTENT with the
regulation graph

BN Semantics – State Space

The set of states is the Cartesian product $\mathcal{B}^n = \{0, 1\}^n$

where n is the number of Boolean variables

We assume the variables to be indexed, then states can be written as vectors

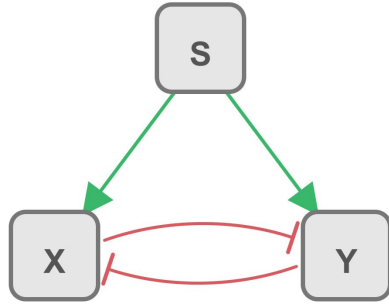


$$(1, 0, 0) \in \mathcal{B}^3$$

S X Y

Dotted lines connect the components of the vector (1, 0, 0) to the variable labels S, X, and Y below them.

BN Semantics – State Transition Graph (STG)



$f_X(S, Y) := S \text{ **AND** not } Y$

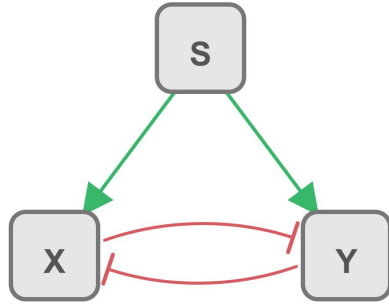
$f_Y(S, X) := S \text{ **AND** not } X$

$f_S := 1$

Consider a state (1, 0, 0)

– to which states the individual variable updates can lead?

BN Semantics – State Transition Graph (STG)



$f_X(S, Y) := S \text{ **AND** not } Y$

$f_Y(S, X) := S \text{ **AND** not } X$

$f_S := 1$

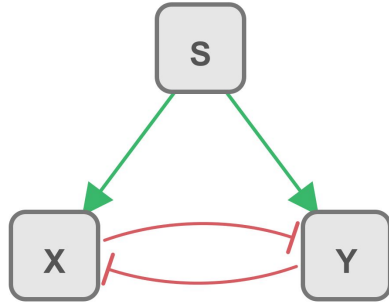
Consider a state (1, 0, 0)

– to which states the individual variable updates can lead?

(1, 1, 0) – by applying f_X

(1, 0, 1) – by applying f_Y

BN Semantics – State Transition Graph (STG)



$f_X(S, Y) := S \text{ **AND** not } Y$

$f_Y(S, X) := S \text{ **AND** not } X$

$f_S := 1$



Consider a state (1, 0, 0)

– to which states the individual variable updates can lead?

(1, 1, 0) – by applying f_X

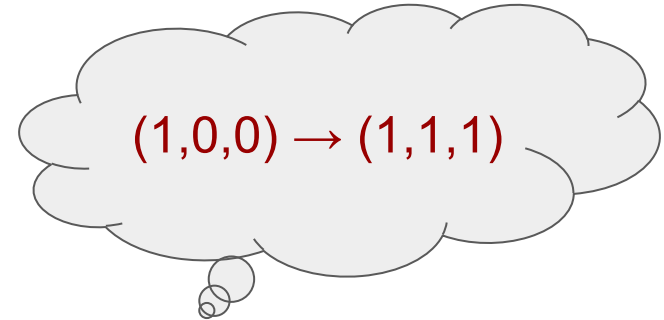
(1, 0, 1) – by applying f_Y

BN Semantics – Update Schemes

Deterministic cell population

- single cell view: presence of “many” signal molecules activating transcription from genes X and Y in the “same” time
- assuming transcription is a deterministic process occurring in all cells in the same way
 - no individual behaviour
- **synchronous semantics**

Consider a state $(1, 0, 0)$



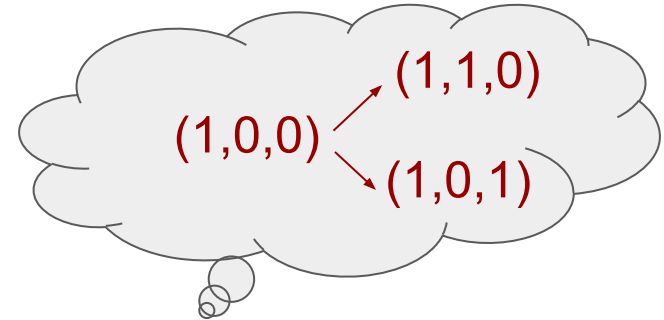
- to which states the individual variable updates can lead?
 - $(1, 1, 0)$ – by applying f_X
 - $(1, 0, 1)$ – by applying f_Y

BN Semantics – Update Schemes

Non-deterministic cell population

- single cell view: signal molecules are in smaller counts and can affect the promoters of genes X and Y in different times
- assuming transcription of every single gene is an atomic event, concurrency allows any possible sequence of atomic events
- **asynchronous semantics**

Consider a state $(1, 0, 0)$



- to which states the individual variable updates can lead?
 - $(1, 1, 0)$ – by applying f_X
 - $(1, 0, 1)$ – by applying f_Y

BN Semantics – Qualitative Understanding of Time

- $X(t + 1) := Y(t)$

- what $(t + 1)$ means?

- it actually means “a next change of the state”

- “next” can be interpreted as “the earliest possible”

BN Semantics – Update Schemes

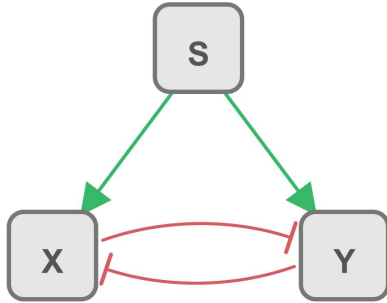
synchronous semantics

- a transition reflects update functions of all variables applied synchronously
- simulation: deterministic, unique sequence of transitions
- produced behaviour may be unrealistic while realistic behaviour can be uncovered

asynchronous semantics

- a transition corresponds to a single variable update
- simulation: non-deterministic, need to make choices to get a sequence of transitions
- conservative approach: model should cover every possible behaviour
(some may occur rarely in real)

BN Semantics – Most Permissive Update

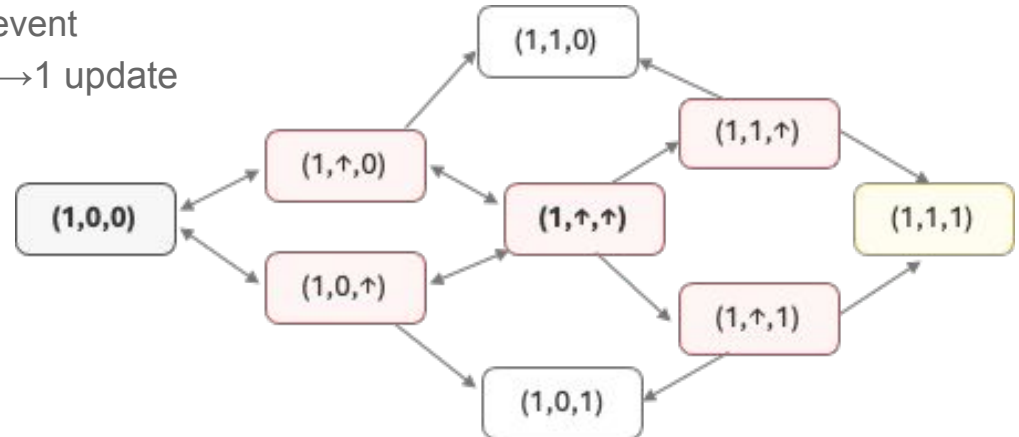


Both activations may be initiated before either has completed.

- considering transcription not to be an atomic event
- introducing transient states: $\uparrow$ = pending $0 \rightarrow 1$ update

Do not choose one update order too early

MP keeps updates “in progress” when timing is unresolved.



Constructing the STG – Reachability

- select an initial state (or a set of initial states – can be all states)
- select a particular update scheme
- apply the update scheme to initial states and then recursively to the reached successor states
- continue until no new (yet unreachable) states appear

Properties of the STG – Paths

- the topology of STG depends on the update mode
 - *synchronous*: there is always a **single transition** from any state
(a path between two states is always unique)
 - *asynchronous* and *MP*: transitions are **branching** leading to several possible paths between two states

Properties of the STG – Attractors

- terminal strongly connected components in the STG
 - represent long-term behaviour
 - single state component – a so-called **steady state (fixed point)**
 - multiple state component can form a cycle representing **oscillation**
or a **complex attractor** with many interfering oscillations
- an attractor cannot be escaped without external change of an appropriate variable value
(we call such external updates **perturbations**)

Properties of the STG – Weak and Strong Basin

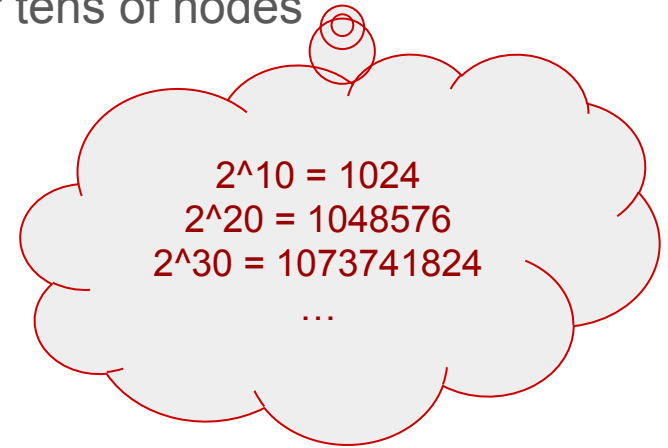
- for a given attractor we can identify all states from which the attractor is reachable, these states form the **attractor basin**
- if there is more than one attractor in the STG, their basins can overlap in some states
- we distinguish two kinds of basins:
 - **weak basin**
(incl. overlapping states from which another attractor can be reached)
 - **strong basin**
(ensuring states from which only the respective attractor is reached)

Properties of the STG – Subspace

- often we look for states where some variables are set to particular values
- a subgraph of STG containing only the selected states and the adequate (projected) transitions forms a **subspace**
- there might be **outgoing transitions** – leading to states not included in a subspace

Algorithms and Tools

- The state space grows exponentially with the number of model variables
- Typical size of models currently used is in lower tens of nodes



- Sophisticated representation of the STG is needed to allow fast operations

Algorithms and Tools

– Aeon/**BIODIVINE**

AEON.py `pip install biodivine_aeon` <https://github.com/sybila/biodivine-aeon-py>

attractor analysis, state space analysis, partially specified models
based on symbolic representation using binary decision diagrams

– BoNesis

`pip install bonesis` <https://bnediction.github.io/bonesis/>

state space analysis: trap space, reachability, **model inference**
based on logic programming

– pyBoolNet, BooN, ...

– outside python: BoolNet, AEON, GinSim, MaBoSS, ...

Model Repos, Communities...

- **BIODIVINE/Boolean Models**

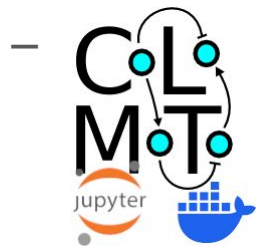
<https://bbm.sybila.fi.muni.cz/>

github-based model repository (find all existing models there...)

- **Cell Collective**

<https://cellcollective.org/>

model repo, annotation, knowledge base, simulation



colomoto.github.io

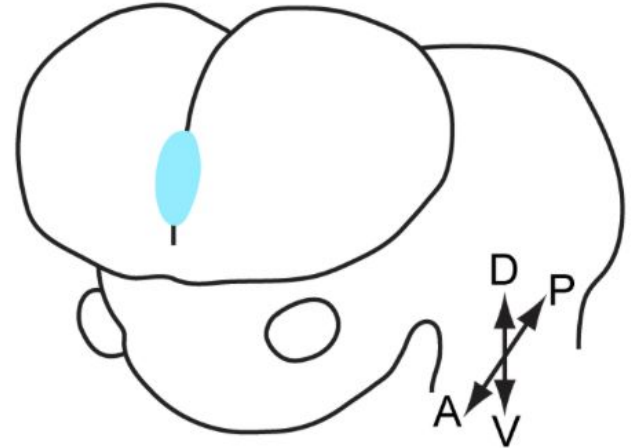
tool interoperability environment, developers community

Model Formats

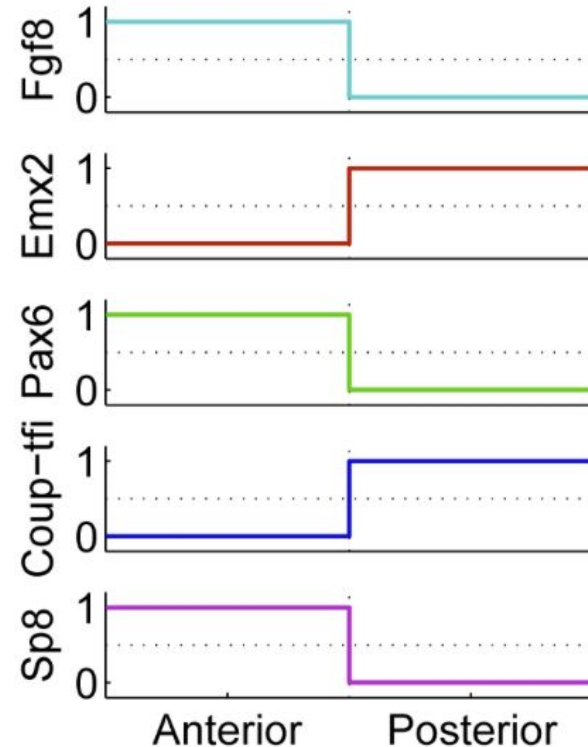
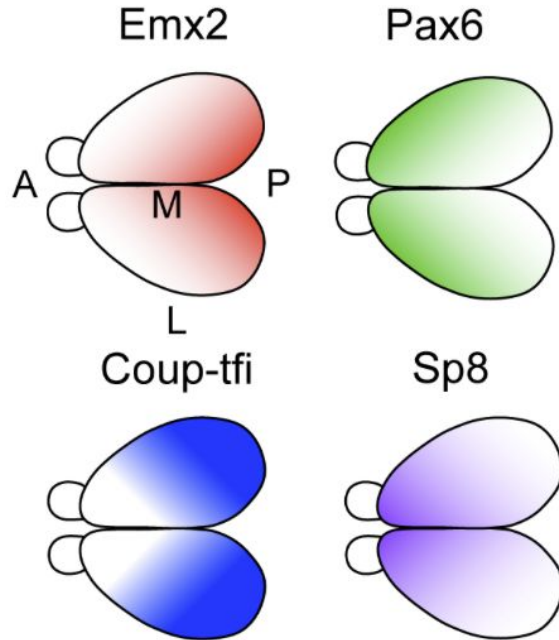
- several formats have been developed around individual tools
(e.g., .aeon, .bnet, .gsim, ...)
- SBML qual standard format <https://sbml.org/documents/specifications/level-3/version-1/qual/>
- most of current tools share at least some of the formats, most of them support SBML to some extent

Example: Mammalian Cortical Area Development Model

- Patterning of the forebrain starts in the anterior neural ridge
- The neural ridge secretes the morphogen FGF8
- FGF8 is hypothesised to diffuse and form a gradient
- Patterning directions:
 - A: anterior, P: posterior, D: dorsal, V: ventral,
 - M: medial, L: lateral



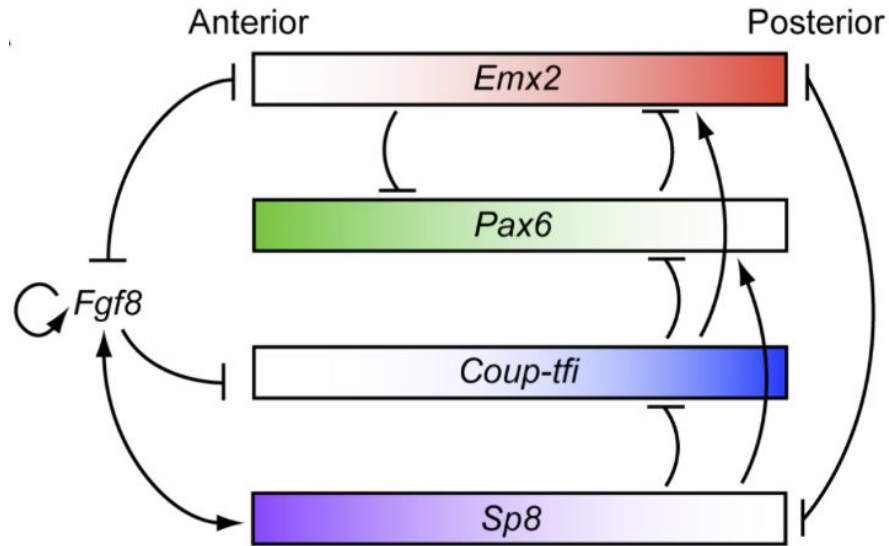
Example: Mammalian Cortical Area Development Model



Mammalian Cortical Area: Modelling Assumptions

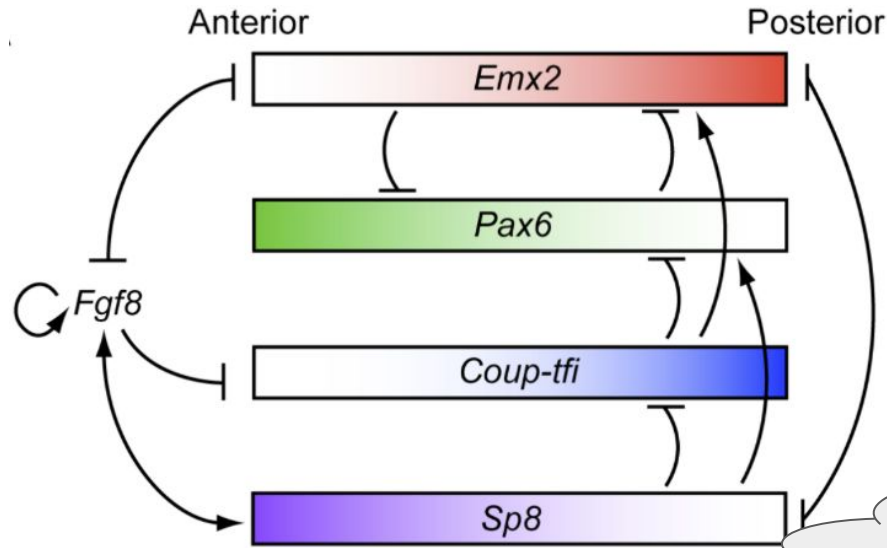
- the entire cell population is assumed to be separated into two compartments:
 - anterior and posterior cells
 - both compartments have the same nodes/regulations
 - separation is solved by different initial conditions
(posterior have zero input FGF8 while anterior have FGF8 input ON by external control)
 - posterior cells are expected to express high Emx2, Coup-tfi gradients
 - anterior cells express highly Pax6 and Sp8
- no signalling communication is considered among cells

Example: Mammalian Cortical Area Development Model



$$\begin{aligned}F(t+1) &= F(t) \& \text{NOT } E(t) \& S(t) \\E(t+1) &= \text{NOT } F(t) \& \text{NOT } P(t) \& C(t) \\&\quad \& \text{NOT } S(t) \\P(t+1) &= \text{NOT } E(t) \& \text{NOT } C(t) \& S(t) \\C(t+1) &= \text{NOT } F(t) \& \text{NOT } S(t) \\S(t+1) &= F(t) \& \text{NOT } E(t)\end{aligned}$$

Example: Mammalian Cortical Area Development Model



$$\begin{aligned} F(t+1) &= F(t) \& \text{NOT } E(t) \& S(t) \\ E(t+1) &= \text{NOT } F(t) \& \text{NOT } P(t) \& C(t) \\ &\quad \& \text{NOT } S(t) \\ P(t+1) &= \text{NOT } E(t) \& \text{NOT } C(t) \& S(t) \\ C(t+1) &= \text{NOT } F(t) \& \text{NOT } S(t) \\ S(t+1) &= F(t) \& \text{NOT } E(t) \end{aligned}$$

simplistic settings of the logics
(only AND operator is considered)

Cortical Area Development: Modelling Problems

- most of the regulations are only hypothesized
- two main expected phenotypes can be achieved:
 - anterior: FGF8, Pax6, Sp8 expressed
 - posterior: Emx2 and Coup-tfi expressed

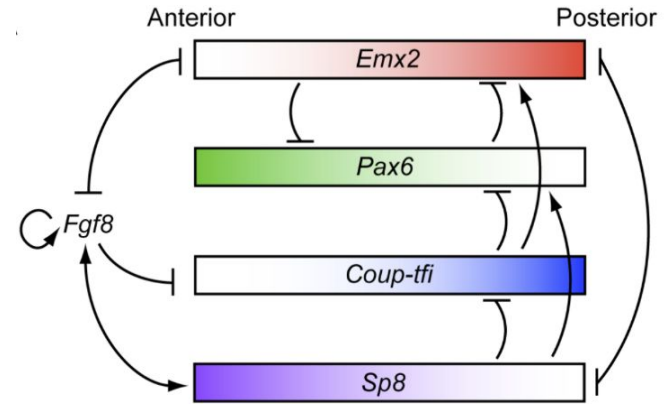
A problem to solve:

Does the model show each phenotype is *robustly* reachable from a corresponding starting state?

FGF8==1, other nodes 0 – for anterior starting state

all nodes 0 – for posterior starting state

If not, how to find a model that shows that?



Hands On

Run the jupyter notebook supporting the lecture: [02-boolean-network-inference.ipynb](#)

Installation instructions for the corresponding Python environment are given in README (see the github folder <https://github.com/sybila/bioinformatics-summer-school/tree/main/bn>)